

std::integral_constant with a Deduced Value Type

ISO/IEC JTC1 SC22 WG21

Document Number: P0377R0

Audience: Library Evolution Working Group

Matt Calabrese (metaprogrammingtheworld@gmail.com)

2016-05-29

Abstract

Following EWG's acceptance of P0127, "Declaring non-type template arguments with auto," [\[1\]](#) this paper proposes a form of `std::integral_constant` that uses the new language feature such that the value type does not need to be explicitly specified by users.

Motivation

Usage of `std::integral_constant` can be redundant in that both an explicit value type needs to be specified and a value of that type. In practice, now the value type can be deduced from the value that is specified although there was no way to do this prior to the acceptance of P0127. Already, a standard alias for `bool` values has been accepted called `std::bool_constant` [\[2\]](#), and Boost.MPL has long provided convenience wrappers for constants of other various built-in types, such as `int_`, `char_`, and `long_` [\[3\]](#). While a form of `std::integral_constant` template with a deduced value type would not completely obviate the utility of such named facilities, it would satisfy many uses of them and remove a little bit of redundancy.

Suggested Specification

There are several ways that such a facility could be provided, each with their own subtleties. The specification that is recommended here is a conservative one where `std::constant` is an alias of `std::integral_constant`:

```
template <auto V>
using constant = integral_constant<decltype(V), V>;
```

The above definition does not change the definition of `std::integral_constant`, so existing uses of `std::integral_constant` cannot break. Because `std::constant` would be an alias of `std::integral_constant`, developers can safely use it for convenience in most places that `std::integral_constant` is required.

Limitations

This formulation of `std::constant` has a limitation. First, it cannot be used to form a `std::integral_constant` whose first argument is a reference type. This limitation could be removed by specifying `std::constant` to have a `decltype(auto)` parameter instead of an `auto` parameter, though that would make it possible for users to accidentally specify a reference. It is unclear how unfortunate this limitation may be, as use of `std::integral_constant` with reference types may be considered by many to be a misuse, and if the type were automatically deduced to be a reference type, it would likely be a programmer error anyway.

Alternative Specifications

The author of the underlying language facility has expressed that in an ideal world, he feels `std::constant` should be the primary template, with `std::integral_constant` reformulated to be an alias of `std::constant`. This option should be considered, although it does risk breaking code, such as code that uses `std::integral_constant` to create references. If this breakage is considered acceptable, then this option should be reconsidered.

Acknowledgments

Thanks to James Touton for proposing the underlying language feature and for his feedback on this paper.

References

[1] James Touton, Mike Spertus: "Declaring non-type template arguments with auto" P0088R0 <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0127r1.html>

[2] Zhihao Yuan: "Wording for `bool_constant`" <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4389.html>

[3] Aleksey Gurtovoy and David Abrahams: "The MPL Reference Manual" http://www.boost.org/doc/libs/1_61_0/libs/mpl/doc/refmanual/numeric.html